



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

## ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

### Лабораторная работа № 3

на тему:

«Исследование равновесий по Нэшу, Парето-оптимальных  
ситуаций и смешанных стратегий в биматричных играх»

Вариант 13

Студент:

ИУ8-105  
(Группа)

Б. Е. Шекунов  
(И. О. Фамилия)

Преподаватель:

Н.С. Коннова  
(И.О. Фамилия)

Москва, 2026 г.

### **Цель работы:**

Изучить критерии выбора стратегий в неантагонистической бескоалиционной игре двух игроков на основе равновесия Нэша и оптимальности по Парето. Проверить эти критерии на примере классических игр. Исследовать свойства оптимальных решений неантагонистических бескоалиционных игр на примере биматричных (2×2)-игр.

### **Постановка задачи:**

1. Сгенерировать случайную биматричную игру (10 x 10). Найти ситуации, равновесные по Нэшу и оптимальные по Парето, а также пересечение множеств этих ситуаций. Выполнить проверку реализованных алгоритмов на примере трех известных игр: «Семейный спор», «Перекресток», «Дилемма заключенного».

2. Для заданной биматричной (2 x 2)-игры, пользуясь теоремами о свойствах оптимальных решений, найти ситуации, равновесные по Нэшу, для исходной игры и для ее смешанного расширения.

### **Исходные данные:**

Исходная биматричная игра в соответствии с вариантом:

$$\begin{bmatrix} (4,1) & (6,2) \\ (11,7) & (0,5) \end{bmatrix}$$

### **Ход работы:**

Ситуация называется равновесной по Нэшу, если ни одному игроку не выгодно в одностороннем порядке отклоняться от выбранной стратегии при фиксированных стратегиях остальных игроков.

Для биматричной игры это означает, что в выбранной клетке:

- выигрыш первого игрока максимален в своём столбце;
- выигрыш второго игрока максимален в своей строке.

Ситуация является оптимальной по Парето, если не существует другой ситуации, в которой выигрыши всех игроков не меньше, а хотя бы одного игрока строго больше.

Для невырожденной биматричной игры  $2 \times 2$ , если существует вполне смешанная ситуация равновесия, то она единственна и вычисляется по формулам:

$$x = v_2 u B^{-1}, \quad y = v_1 A^{-1} u,$$

где

$$v_1 = \frac{1}{u A^{-1} u}, \quad v_2 = \frac{1}{u B^{-1} u}.$$

Возможны три случая:

1. Если хотя бы один игрок имеет строго доминирующую стратегию, то игра и её смешанное расширение имеют единственную равновесную ситуацию по Нэшу.
2. Если в игре нет равновесия по Нэшу в чистых стратегиях, то в смешанном расширении существует вполне смешанная ситуация равновесия.
3. Если игра имеет две равновесные ситуации по Нэшу в чистых стратегиях, то в смешанном дополнении существует ещё одна вполне смешанная ситуация равновесия.

Дальше рассмотрим различные ситуации на конкретных примерах.

Для проверки корректности сначала разобраны классические игры, результат которых известен заранее, потом разобрана Случайная биматричная игра  $10 \times 10$  и игра, заданная по варианту.

Для удобного отображения на рисунках ситуации равновесные по Нэшу, оптимальные по Парето и их пересечения обозначены различными цветами.

Легенда:

|    |               |
|----|---------------|
| N  | – Нэш         |
| P  | – Парето      |
| NP | – Пересечение |

Рисунок 1 – цвета для обозначения в таблицах

## Проверка алгоритма на классических играх

### 1. Дилемма заключённого

Матрица игры (в клетке: (A<sub>ij</sub>, B<sub>ij</sub>) и метка результата)

|          | Говорить   | Молчать    |
|----------|------------|------------|
| Говорить | (-5, -5) N | (0, -10) P |
| Молчать  | (-10, 0) P | (-1, -1) P |

Краткая сводка:

- Нэш: (Говорить, Говорить) = (-5, -5)
- Парето: (Говорить, Молчать) = (0, -10); (Молчать, Говорить) = (-10, 0); (Молчать, Молчать) = (-1, -1)
- Пересечение: нет
- Итоговый выбор: (Говорить, Говорить) = (-5, -5)

Смешанные стратегии

У игрока 1 есть строго доминирующая стратегия: Говорить.

У игрока 2 есть строго доминирующая стратегия: Говорить.

Чистые равновесия по Нэшу:

(Говорить, Говорить) = (-5, -5)

Полностью смешанная равновесная ситуация: не существует.

### *Рисунок 2 – Дилемма заключённого*

В данной игре единственная устойчивая ситуация по Нэшу не является оптимальной по Парето. Это и отражает парадокс дилеммы заключённого: индивидуально рациональное поведение приводит к худшему совместному результату.

Для смешанных стратегий вполне смешанной равновесной ситуации не существует, поскольку у обоих игроков имеется строго доминирующая стратегия «Говорить».

## 2. Семейный спор

Матрица игры (в клетке:  $(A_{ij}, B_{ij})$  и метка результата)

|        | Футбол    | Театр     |
|--------|-----------|-----------|
| Футбол | (4, 1) NP | (0, 0)    |
| Театр  | (0, 0)    | (1, 4) NP |

Краткая сводка:

- Нэш: (Футбол, Футбол) = (4, 1); (Театр, Театр) = (1, 4)
- Парето: (Футбол, Футбол) = (4, 1); (Театр, Театр) = (1, 4)
- Пересечение: (Футбол, Футбол) = (4, 1); (Театр, Театр) = (1, 4)
- Итоговый выбор: (Футбол, Футбол) = (4, 1); (Театр, Театр) = (1, 4)

Смешанные стратегии

У игрока 1 строго доминирующей стратегии нет.

У игрока 2 строго доминирующей стратегии нет.

Чистые равновесия по Нэшу:

(Футбол, Футбол) = (4, 1); (Театр, Театр) = (1, 4)

Полностью смешанная равновесная ситуация существует.

Знаменатель для q: 5

Знаменатель для p: 5

$x = [p, 1-p] = [0.8, 0.2]$ ,  $y = [q, 1-q] = [0.2, 0.8]$

Проверка игрока 1:  $v1(row1) = 0.8$ ,  $v1(row2) = 0.8$

Проверка игрока 2:  $v2(col1) = 0.8$ ,  $v2(col2) = 0.8$

Равновесные выигрыши:  $v1 = 0.8$ ,  $v2 = 0.8$

Проверка по формуле из методички:

$\det(A) = 4$ ,  $\det(B) = 4$

$A^{-1} = \begin{bmatrix} 0.25 & 0 \\ 0 & 1 \end{bmatrix}$

$B^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 0.25 \end{bmatrix}$

$v1 = 1 / (u * A^{-1} * u) = 0.8$ ,  $v2 = 1 / (u * B^{-1} * u) = 0.8$

$x = v2 * u * B^{-1} = [0.8, 0.2]$

$y = v1 * A^{-1} * u = [0.2, 0.8]$

*Рисунок 3 – Семейный спор*

В этой игре существует два равновесия в чистых стратегиях, и оба они одновременно являются Парето-оптимальными.

Кроме того, существует смешанная равновесная ситуация:

$$x = [0.8; 0.2], \quad y = [0.2; 0.8]$$

Равновесные выигрыши:

$$v1 = 0.8, \quad v2 = 0.8.$$

### 3. Перекрёсток

Матрица игры (в клетке: (A<sub>ij</sub>, B<sub>ij</sub>) и метка результата)

|       | Стоп        | Ехать       |
|-------|-------------|-------------|
| Стоп  | (1, 1) P    | (0.7, 2) NP |
| Ехать | (2, 0.5) NP | (-2, -12)   |

Краткая сводка:

- Нэш: (Стоп, Ехать) = (0.7, 2); (Ехать, Стоп) = (2, 0.5)
- Парето: (Стоп, Стоп) = (1, 1); (Стоп, Ехать) = (0.7, 2); (Ехать, Стоп) = (2, 0.5)
- Пересечение: (Стоп, Ехать) = (0.7, 2); (Ехать, Стоп) = (2, 0.5)
- Итоговый выбор: (Стоп, Ехать) = (0.7, 2); (Ехать, Стоп) = (2, 0.5)

Смешанные стратегии

У игрока 1 строго доминирующей стратегии нет.

У игрока 2 строго доминирующей стратегии нет.

Чистые равновесия по Нэшу:

(Стоп, Ехать) = (0.7, 2); (Ехать, Стоп) = (2, 0.5)

Полностью смешанная равновесная ситуация существует.

Знаменатель для q: -3.7

Знаменатель для p: -13.5

$x = [p, 1-p] = [0.926, 0.074]$ ,  $y = [q, 1-q] = [0.73, 0.27]$

Проверка игрока 1:  $v1(row1) = 0.919$ ,  $v1(row2) = 0.919$

Проверка игрока 2:  $v2(col1) = 0.963$ ,  $v2(col2) = 0.963$

Равновесные выигрыши:  $v1 = 0.919$ ,  $v2 = 0.963$

Проверка по формуле из методички:

$\det(A) = -3.4$ ,  $\det(B) = -13$

$A^{(-1)} = \begin{bmatrix} 0.588 & 0.206 \\ 0.588 & -0.294 \end{bmatrix}$

$B^{(-1)} = \begin{bmatrix} 0.923 & 0.154 \\ 0.038 & -0.077 \end{bmatrix}$

$v1 = 1 / (u * A^{(-1)} * u) = 0.919$ ,  $v2 = 1 / (u * B^{(-1)} * u) = 0.963$

$x = v2 * u * B^{(-1)} = [0.926, 0.074]$

$y = v1 * A^{(-1)} * u = [0.73, 0.27]$

Рисунок 4 – Перекрёсток

Устойчивыми являются две асимметричные ситуации, в которых один игрок уступает дорогу, а другой проезжает перекрёсток.

По общему виду это согласуется с классической игрой «Перекрёсток», где также имеются два равновесия по Нэшу и три Парето-оптимальные ситуации.

Смешанная равновесная ситуация:

$$\begin{aligned}x &= [0.926; 0.074], & y &= [0.73; 0.27], \\v1 &= 0.919, & v2 &= 0.963.\end{aligned}$$

## Случайная биматричная игра 10×10

Матрица игры (в клетке: (A<sub>ij</sub>, B<sub>ij</sub>) и метка результата)

|     | B1         | B2        | B3         | B4         | B5         | B6         | B7         | B8          | B9         | B10        |
|-----|------------|-----------|------------|------------|------------|------------|------------|-------------|------------|------------|
| A1  | (-32, -10) | (10, -40) | (44, 24)   | (-14, 9)   | (0, 45) P  | (-47, -30) | (-9, -9)   | (-44, 32)   | (-28, -1)  | (22, 1)    |
| A2  | (-21, -35) | (31, 4)   | (18, -49)  | (-4, 43)   | (37, 29)   | (6, 9)     | (29, -37)  | (-26, 20)   | (6, -32)   | (-43, 32)  |
| A3  | (-15, -1)  | (33, -37) | (-20, 43)  | (-37, -3)  | (29, -2)   | (-10, -1)  | (-4, 1)    | (-30, 38)   | (46, -41)  | (47, -42)  |
| A4  | (46, 4)    | (40, -5)  | (-27, -2)  | (38, -9)   | (-37, 41)  | (-47, -47) | (-24, 20)  | (-10, -44)  | (24, -44)  | (-50, -17) |
| A5  | (3, -49)   | (-5, 6)   | (31, -20)  | (-35, 30)  | (-18, -25) | (31, 37) N | (46, 15)   | (-22, -48)  | (44, 16)   | (2, -18)   |
| A6  | (36, -16)  | (-3, 17)  | (-16, 25)  | (-23, 32)  | (16, 42)   | (-16, -8)  | (-20, -38) | (37, 37)    | (-10, -21) | (-7, -34)  |
| A7  | (25, -13)  | (-7, -26) | (40, -43)  | (-45, -18) | (-17, 17)  | (-45, 36)  | (0, -32)   | (43, 44) NP | (29, 3)    | (-18, -45) |
| A8  | (27, 5)    | (-5, 13)  | (-35, -10) | (-24, -26) | (37, -28)  | (23, 2)    | (-14, 16)  | (-19, -30)  | (8, 42)    | (49, 26)   |
| A9  | (-29, -44) | (50, -11) | (32, 31)   | (-31, -6)  | (18, -17)  | (-43, -28) | (-4, 48) P | (38, 15)    | (47, -24)  | (-40, 26)  |
| A10 | (48, -14)  | (6, 26)   | (50, 26) P | (40, 7)    | (0, -31)   | (-46, -47) | (-33, 39)  | (16, -15)   | (1, 3)     | (-6, -23)  |

Краткая сводка:

- Нэш: (A5, B6) = (31, 37); (A7, B8) = (43, 44)
- Парето: (A1, B5) = (0, 45); (A7, B8) = (43, 44); (A9, B7) = (-4, 48); (A10, B3) = (50, 26)
- Пересечение: (A7, B8) = (43, 44)
- Итоговый выбор: (A7, B8) = (43, 44)

Смешанные стратегии: анализ выполняется только для игр 2x2.

### Рисунок 5 – Случайная биматричная игра 10×10

В случайной сгенерированной биматричной игре существуют все 3 возможные типа ситуаций.

Равновесные только по Нэшу:

$$(A5, B6) = (31, 37),$$

Оптимальные только по Парето:

$$(A1, B5) = (0, 45),$$

$$(A9, B7) = (-4, 48),$$

$$(A10, B3) = (50, 26)$$

Одновременно равновесные по Нэшу, и Парето-оптимальные:

$$(A7, B8) = (43, 44).$$

## Исследование игры в соответствии с вариантом

Матрица игры (в клетке: (A<sub>ij</sub>, B<sub>ij</sub>) и метка результата)

|            | $\beta 1$  | $\beta 2$ |
|------------|------------|-----------|
| $\alpha 1$ | (4, 1)     | (6, 2) N  |
| $\alpha 2$ | (11, 7) NP | (0, 5)    |

Краткая сводка:

- Нэш: ( $\alpha 1$ ,  $\beta 2$ ) = (6, 2); ( $\alpha 2$ ,  $\beta 1$ ) = (11, 7)
- Парето: ( $\alpha 2$ ,  $\beta 1$ ) = (11, 7)
- Пересечение: ( $\alpha 2$ ,  $\beta 1$ ) = (11, 7)
- Итоговый выбор: ( $\alpha 2$ ,  $\beta 1$ ) = (11, 7)

Смешанные стратегии

У игрока 1 строго доминирующей стратегии нет.

У игрока 2 строго доминирующей стратегии нет.

Чистые равновесия по Нэшу:

( $\alpha 1$ ,  $\beta 2$ ) = (6, 2); ( $\alpha 2$ ,  $\beta 1$ ) = (11, 7)

Полностью смешанная равновесная ситуация существует.

Знаменатель для q: -13

Знаменатель для p: -3

$x = [p, 1-p] = [0.667, 0.333]$ ,  $y = [q, 1-q] = [0.462, 0.538]$

Проверка игрока 1:  $v1(row1) = 5.077$ ,  $v1(row2) = 5.077$

Проверка игрока 2:  $v2(col1) = 3$ ,  $v2(col2) = 3$

Равновесные выигрыши:  $v1 = 5.077$ ,  $v2 = 3$

Проверка по формуле из методички:

$\det(A) = -66$ ,  $\det(B) = -9$

$A^{(-1)} = [[0, 0.091], [0.167, -0.061]]$

$B^{(-1)} = [[-0.556, 0.222], [0.778, -0.111]]$

$v1 = 1 / (u * A^{(-1)} * u) = 5.077$ ,  $v2 = 1 / (u * B^{(-1)} * u) = 3$

$x = v2 * u * B^{(-1)} = [0.667, 0.333]$

$y = v1 * A^{(-1)} * u = [0.462, 0.538]$

*Рисунок 6 – Исследование игры в соответствии с вариантом*

Игра имеет два равновесия по Нэшу в чистых стратегиях:

$$(\alpha 1, \beta 2) = (6, 2)$$

$$(\alpha 2, \beta 1) = (11, 7)$$

Парето-оптимальной ситуацией является:

$$(\alpha 2, \beta 1) = (11, 7)$$

Эта ситуация доминирует все остальные по совместному результату, так как даёт наибольший выигрыш обоим игрокам одновременно.

Пересечение множества равновесий по Нэшу и множества Парето-оптимальных ситуаций состоит из одной точки. Следовательно, именно эта ситуация является наилучшим итоговым выбором по двум критериям одновременно.

Так как в исходной игре имеются две равновесные ситуации по Нэшу в чистых стратегиях, в смешанном дополнении должна существовать ещё одна вполне смешанная ситуация равновесия.

В данном случае это следующая ситуация:

$$x = [0.667; 0.333], y = [0.462; 0.538].$$

Равновесные выигрыши:

$$v_1 = 5.077, \quad v_2 = 3.$$

### **Вывод:**

В ходе лабораторной работы были изучены два критерия выбора решений в неантагонистических бескоалиционных играх: равновесие по Нэшу и оптимальность по Парето.

Было показано, что множества равновесных по Нэшу и Парето-оптимальных ситуаций могут:

- совпадать полностью, как в игре «Семейный спор»;
- пересекаться частично, как в игре «Перекрёсток»;
- не пересекаться вовсе, как в дилемме заключённого.

Для случайной игры  $10 \times 10$  была найдена ситуация, одновременно равновесная по Нэшу и оптимальная по Парето:

$$(A7, B8) = (43, 44).$$

Для игры из варианта найдены:

- два равновесия по Нэшу в чистых стратегиях:

$$(\alpha_1, \beta_2) = (6, 2), \quad (\alpha_2, \beta_1) = (11, 7);$$

- одна Парето-оптимальная ситуация:

$$(\alpha 2, \beta 1) = (11, 7);$$

- одна ситуация в пересечении двух множеств:

$$(\alpha 2, \beta 1) = (11, 7).$$

В смешанном расширении игры варианта 13 найдена вполне смешанная ситуация равновесия:

$$x = [0.667; 0.333], \quad y = [0.462; 0.538],$$

с равновесными выигрышами

$$v1 = 5.077, \quad v2 = 3.$$

Таким образом, алгоритмы поиска равновесия по Нэшу, Парето-оптимальных ситуаций и смешанных равновесий были реализованы и проверены на классических примерах, случайной игре и индивидуальном варианте.

## Приложение (исходный код)

```
from __future__ import annotations

import random
from typing import Dict, List, Optional, Sequence, Tuple

Number = float
Matrix = List[List[Number]]
Position = Tuple[int, int]

RANDOM_GAME_SEED = 567
RANDOM_GAME_SIZE = 10
RANDOM_GAME_LOW = -50
RANDOM_GAME_HIGH = 50

class Ansi:
    RESET = "\033[0m"
    BOLD = "\033[1m"
    BLUE_BG = "\033[44;97m"
    GREEN_BG = "\033[42;30m"
    MAGENTA_BG = "\033[45;97m"
    CYAN = "\033[36m"
    YELLOW = "\033[33m"

def colorize(text: str, style: str) -> str:
    if not style:
        return text
    return f"{style}{text}{Ansi.RESET}"

CLASSIC_GAMES = {
    "Дилемма заключённого": {
        "rows": ["Говорить", "Молчать"],
        "cols": ["Говорить", "Молчать"],
        "pairs": [
            [(-5, -5), (0, -10)],
            [(-10, 0), (-1, -1)],
        ],
        "comment": "Устойчивой по Нэшу считается только ситуация (Г, Г).",
    },
    "Семейный спор": {
        "rows": ["Футбол", "Театр"],
        "cols": ["Футбол", "Театр"],
        "pairs": [
            [(4, 1), (0, 0)],
            [(0, 0), (1, 4)],
        ],
        "comment": "Классическая игра с двумя чистыми и одной смешанной равновесной ситуацией.",
    },
    "Перекрёсток (Феррари / КамАЗ)": {
        "rows": ["Стоп", "Ехать"],
        "cols": ["Стоп", "Ехать"],
        "pairs": [
            [(1.0, 1.0), (0.7, 2.0)],
            [(2.0, 0.5), (-2, -12)],
        ],
        "comment": "Несимметричный учебный вариант перекрёстка для проверки алгоритма.",
    },
}

def fmt(x: Number) -> str:
    if abs(x - round(x)) < 1e-9:
        return str(int(round(x)))
    return f"{x:.3f}".rstrip("0").rstrip(".")

def pair_str(a: Number, b: Number) -> str:
    return f"({fmt(a)}, {fmt(b)})"

def pairs_to_matrices(pairs: Sequence[Sequence[Tuple[Number, Number]]]) -> Tuple[Matrix, Matrix]:
    a = [[cell[0] for cell in row] for row in pairs]
    b = [[cell[1] for cell in row] for row in pairs]
    return a, b

def size_of(matrix: Matrix) -> Tuple[int, int]:
    return len(matrix), len(matrix[0])
```

```

def print_separator(char: str = "=", width: int = 96) -> None:
    print(char * width)

def positions_to_text(
    positions: Sequence[Position],
    pairs: Sequence[Sequence[Tuple[Number, Number]]],
    row_labels: Optional[Sequence[str]] = None,
    col_labels: Optional[Sequence[str]] = None,
) -> str:
    if not positions:
        return "net"
    parts = []
    for i, j in positions:
        r_name = row_labels[i] if row_labels else f"A{i + 1}"
        c_name = col_labels[j] if col_labels else f"B{j + 1}"
        parts.append(f"({r_name}, {c_name}) = {pair_str(*pairs[i][j])}")
    return "; ".join(parts)

def intersection(xs: Sequence[Position], ys: Sequence[Position]) -> List[Position]:
    return sorted(set(xs) & set(ys))

def find_nash_equilibria(a: Matrix, b: Matrix, strict: bool = False) -> List[Position]:
    m, n = size_of(a)
    result = []
    for i in range(m):
        for j in range(n):
            if strict:
                row_ok = all(a[i][j] > a[k][j] for k in range(m) if k != i)
                col_ok = all(b[i][j] > b[i][l] for l in range(n) if l != j)
            else:
                row_ok = all(a[i][j] >= a[k][j] for k in range(m) if k != i)
                col_ok = all(b[i][j] >= b[i][l] for l in range(n) if l != j)
            if row_ok and col_ok:
                result.append((i, j))
    return result

def find_pareto_strong(a: Matrix, b: Matrix) -> List[Position]:
    m, n = size_of(a)
    result = []
    for i in range(m):
        for j in range(n):
            dominated = False
            for p in range(m):
                for q in range(n):
                    better_or_equal_for_both = a[p][q] >= a[i][j] and b[p][q] >= b[i][j]
                    strictly_better_for_someone = a[p][q] > a[i][j] or b[p][q] > b[i][j]
                    if better_or_equal_for_both and strictly_better_for_someone:
                        dominated = True
                        break
                if dominated:
                    break
            if not dominated:
                result.append((i, j))
    return result

def find_pareto_weak(a: Matrix, b: Matrix) -> List[Position]:
    m, n = size_of(a)
    result = []
    for i in range(m):
        for j in range(n):
            strongly_dominated = False
            for p in range(m):
                for q in range(n):
                    if a[p][q] > a[i][j] and b[p][q] > b[i][j]:
                        strongly_dominated = True
                        break
                if strongly_dominated:
                    break
            if not strongly_dominated:
                result.append((i, j))
    return result

def cell_marker(pos: Position, nash: set[Position], pareto: set[Position]) -> str:
    in_nash = pos in nash
    in_pareto = pos in pareto

```

```

    if in_nash and in_pareto:
        return "NP"
    if in_nash:
        return "N"
    if in_pareto:
        return "P"
    return " "

def cell_style(pos: Position, nash: set[Position], pareto: set[Position]) -> str:
    in_nash = pos in nash
    in_pareto = pos in pareto
    if in_nash and in_pareto:
        return Ansi.MAGENTA_BG
    if in_nash:
        return Ansi.BLUE_BG
    if in_pareto:
        return Ansi.GREEN_BG
    return ""

def print_legend() -> None:
    print("Легенда:")
    print(" " + colorize(" N ", Ansi.BLUE_BG) + " – строгий Нэш")
    print(" " + colorize(" P ", Ansi.GREEN_BG) + " – сильный Парето")
    print(" " + colorize(" NP ", Ansi.MAGENTA_BG) + " – пересечение")
    print()

def print_colored_matrix(
    pairs: Sequence[Sequence[Tuple[Number, Number]]],
    nash: Sequence[Position],
    pareto: Sequence[Position],
    row_labels: Optional[Sequence[str]] = None,
    col_labels: Optional[Sequence[str]] = None,
) -> None:
    m = len(pairs)
    n = len(pairs[0])
    if row_labels is None:
        row_labels = [f"A{i + 1}" for i in range(m)]
    if col_labels is None:
        col_labels = [f"B{j + 1}" for j in range(n)]

    nash_set = set(nash)
    pareto_set = set(pareto)

    widths = []
    for j in range(n):
        max_len = len(str(col_labels[j]))
        for i in range(m):
            marker = cell_marker((i, j), nash_set, pareto_set)
            cell_text = f"{pair_str(*pairs[i][j])} {marker}".rstrip()
            max_len = max(max_len, len(cell_text))
        widths.append(max_len + 2)

    first_col_w = max(len(max(row_labels, key=len)), 8)
    header = " " * (first_col_w + 3)
    for j in range(n):
        header += f"{str(col_labels[j]):^{widths[j]}}"
    print(header)

    for i in range(m):
        line = f"{row_labels[i]:>{first_col_w}} |"
        for j in range(n):
            marker = cell_marker((i, j), nash_set, pareto_set)
            cell_text = f"{pair_str(*pairs[i][j])} {marker}".rstrip()
            padded = f"{cell_text:^{widths[j]}}"
            style = cell_style((i, j), nash_set, pareto_set)
            line += colorize(padded, style) if style else padded
        print(line)
    print()

def has_strictly_dominant_row_strategy(a: Matrix) -> Optional[int]:
    m, n = size_of(a)
    for i in range(m):
        dominates_all = True
        for k in range(m):
            if k == i:
                continue

```

```

        for j in range(n):
            if not (a[i][j] > a[k][j]):
                dominates_all = False
                break
        if not dominates_all:
            break
    if dominates_all:
        return i
    return None

def has_strictly_dominant_col_strategy(b: Matrix) -> Optional[int]:
    m, n = size_of(b)
    for j in range(n):
        dominates_all = True
        for l in range(n):
            if l == j:
                continue
            for i in range(m):
                if not (b[i][j] > b[i][l]):
                    dominates_all = False
                    break
            if not dominates_all:
                break
        if dominates_all:
            return j
    return None

def inverse_2x2(mtx: Matrix) -> Tuple[Optional[Matrix], Number]:
    a, b = mtx[0]
    c, d = mtx[1]
    det = a * d - b * c
    if abs(det) < 1e-12:
        return None, det
    inv = [[d / det, -b / det], [-c / det, a / det]]
    return inv, det

def mixed_equilibrium_2x2(a: Matrix, b: Matrix) -> Dict[str, object]:
    a11, a12 = a[0]
    a21, a22 = a[1]
    b11, b12 = b[0]
    b21, b22 = b[1]

    result: Dict[str, object] = {"exists": False, "reason": None}

    denom_q = a11 - a12 - a21 + a22
    denom_p = b11 - b12 - b21 + b22
    result["denom_q"] = denom_q
    result["denom_p"] = denom_p

    if abs(denom_q) < 1e-12 or abs(denom_p) < 1e-12:
        result["reason"] = (
            "Одна из формул безразличия вырождается: знаменатель равен нулю, "
            "поэтому полностью смешанную ситуацию построить нельзя."
        )
        return result

    q = (a22 - a12) / denom_q
    p = (b22 - b21) / denom_p
    x = [p, 1 - p]
    y = [q, 1 - q]

    result["p"] = p
    result["q"] = q
    result["x"] = x
    result["y"] = y

    if not (0 < p < 1 and 0 < q < 1):
        result["reason"] = (
            "Решение уравнений безразличия лежит на границе или вне [0, 1], "
            "поэтому полностью смешанной равновесной ситуации нет."
        )
        return result

    v1_from_row1 = q * a11 + (1 - q) * a12
    v1_from_row2 = q * a21 + (1 - q) * a22
    v2_from_col1 = p * b11 + (1 - p) * b21

```

```

v2_from_col2 = p * b12 + (1 - p) * b22

result["v1_from_row1"] = v1_from_row1
result["v1_from_row2"] = v1_from_row2
result["v2_from_col1"] = v2_from_col1
result["v2_from_col2"] = v2_from_col2
result["v1"] = (v1_from_row1 + v1_from_row2) / 2
result["v2"] = (v2_from_col1 + v2_from_col2) / 2

a_inv, det_a = inverse_2x2(a)
b_inv, det_b = inverse_2x2(b)
result["det_a"] = det_a
result["det_b"] = det_b
result["a_inv"] = a_inv
result["b_inv"] = b_inv

if a_inv is not None and b_inv is not None:
    sum_a_inv = sum(sum(row) for row in a_inv)
    sum_b_inv = sum(sum(row) for row in b_inv)
    result["sum_a_inv"] = sum_a_inv
    result["sum_b_inv"] = sum_b_inv

    if abs(sum_a_inv) > 1e-12 and abs(sum_b_inv) > 1e-12:
        v1_formula = 1 / sum_a_inv
        v2_formula = 1 / sum_b_inv
        y_formula = [
            v1_formula * (a_inv[0][0] + a_inv[0][1]),
            v1_formula * (a_inv[1][0] + a_inv[1][1]),
        ]
        x_formula = [
            v2_formula * (b_inv[0][0] + b_inv[1][0]),
            v2_formula * (b_inv[0][1] + b_inv[1][1]),
        ]
        result["v1_formula"] = v1_formula
        result["v2_formula"] = v2_formula
        result["x_formula"] = x_formula
        result["y_formula"] = y_formula

result["exists"] = True
return result

def print_mixed_analysis(
    a: Matrix,
    b: Matrix,
    pairs: Sequence[Sequence[Tuple[Number, Number]]],
    row_labels: Optional[Sequence[str]] = None,
    col_labels: Optional[Sequence[str]] = None,
) -> None:
    if size_of(a) != (2, 2):
        print("Смешанные стратегии: анализ выполняется только для игр 2x2.\n")
        return

    print("Смешанные стратегии")
    print("-" * 96)

    dom_row = has_strictly_dominant_row_strategy(a)
    dom_col = has_strictly_dominant_col_strategy(b)

    if dom_row is None:
        print("У игрока 1 строго доминирующей стратегии нет.")
    else:
        name = row_labels[dom_row] if row_labels else f"A{dom_row + 1}"
        print(f"У игрока 1 есть строго доминирующая стратегия: {name}.")

    if dom_col is None:
        print("У игрока 2 строго доминирующей стратегии нет.")
    else:
        name = col_labels[dom_col] if col_labels else f"B{dom_col + 1}"
        print(f"У игрока 2 есть строго доминирующая стратегия: {name}.")

    pure_nash = find_nash_equilibria(a, b, strict=False)
    print("Чистые равновесия по Нэшу:")
    print(positions_to_text(pure_nash, pairs, row_labels, col_labels))

    mix = mixed_equilibrium_2x2(a, b)
    if not mix["exists"]:

```

```

        print("Полностью смешанная равновесная ситуация: не существует.")
        print(f"Причина: {mix['reason']}")
        print()
        return

    print("Полностью смешанная равновесная ситуация существует.")
    print(f"Знаменатель для q: {fmt(mix['denom_q'])}")
    print(f"Знаменатель для p: {fmt(mix['denom_p'])}")
    print(
        f"x = [p, 1-p] = [{fmt(mix['x'][0])}, {fmt(mix['x'][1])}], "
        f"y = [q, 1-q] = [{fmt(mix['y'][0])}, {fmt(mix['y'][1])}]"
    )
    print(
        f"Проверка игрока 1: v1(row1) = {fmt(mix['v1_from_row1'])}, "
        f"v1(row2) = {fmt(mix['v1_from_row2'])}"
    )
    print(
        f"Проверка игрока 2: v2(col1) = {fmt(mix['v2_from_col1'])}, "
        f"v2(col2) = {fmt(mix['v2_from_col2'])}"
    )
    print(f"Равновесные выигрыши: v1 = {fmt(mix['v1'])}, v2 = {fmt(mix['v2'])}")

    if mix.get("a_inv") is not None and mix.get("b_inv") is not None:
        print("Проверка по формуле из методички:")
        print(f"det(A) = {fmt(mix['det a'])}, det(B) = {fmt(mix['det b'])}")
        print(
            f"A^(-1) = [{fmt(mix['a_inv'][0][0])}, {fmt(mix['a_inv'][0][1])}, "
            f"[{fmt(mix['a_inv'][1][0])}, {fmt(mix['a_inv'][1][1])}]"
        )
        print(
            f"B^(-1) = [{fmt(mix['b_inv'][0][0])}, {fmt(mix['b_inv'][0][1])}, "
            f"[{fmt(mix['b_inv'][1][0])}, {fmt(mix['b_inv'][1][1])}]"
        )
        if mix.get("v1_formula") is not None and mix.get("v2_formula") is not None:
            print(
                f"v1 = 1 / (u * A^(-1) * u) = {fmt(mix['v1_formula'])}, "
                f"v2 = 1 / (u * B^(-1) * u) = {fmt(mix['v2_formula'])}"
            )
            print(f"x = v2 * u * B^(-1) = [{fmt(mix['x_formula'][0])}, "
                  f"{fmt(mix['x_formula'][1])}]")
            print(f"y = v1 * A^(-1) * u = [{fmt(mix['y_formula'][0])}, "
                  f"{fmt(mix['y_formula'][1])}]")
            print()

def generate_random_bimatrix(
    rows: int,
    cols: int,
    low: int,
    high: int,
    seed: int,
) -> List[List[Tuple[Number, Number]]]:
    rng = random.Random(seed)
    pairs = []
    for _ in range(rows):
        row = []
        for _ in range(cols):
            row.append((rng.randint(low, high), rng.randint(low, high)))
        pairs.append(row)
    return pairs

def analyze_game(
    name: str,
    pairs: Sequence[Sequence[Tuple[Number, Number]]],
    row_labels: Optional[Sequence[str]] = None,
    col_labels: Optional[Sequence[str]] = None,
    extra_comment: Optional[str] = None,
) -> None:
    a, b = pairs_to_matrices(pairs)

    nash_classic = find_nash_equilibria(a, b, strict=False)
    nash_strict = find_nash_equilibria(a, b, strict=True)
    pareto_strong = find_pareto_strong(a, b)
    pareto_weak = find_pareto_weak(a, b)
    inter = intersection(nash_strict, pareto_strong)

    print_separator("=")

```

```

print(name)
print_separator("=")
if extra_comment:
    print(extra_comment)
    print()

print_legend()
print("Матрица игры (в клетке: (A_ij, B_ij) и метка результата)")
print_colored_matrix(pairs, nash_strict, pareto_strong, row_labels, col_labels)

print("Краткая сводка:")
print(f"- Нэш (нестрогий, по лекции): {positions_to_text(nash_classic, pairs, row_labels, col_labels)}")
print(f"- Нэш (строгий, по семинару): {positions_to_text(nash_strict, pairs, row_labels, col_labels)}")
print(f"- Парето (сильный): {positions_to_text(pareto_strong, pairs, row_labels, col_labels)}")
print(f"- Парето (слабый): {positions_to_text(pareto_weak, pairs, row_labels, col_labels)}")
print(f"- Пересечение: {positions_to_text(inter, pairs, row_labels, col_labels)}")

if inter:
    print(f"- Итоговый выбор: {positions_to_text(inter, pairs, row_labels, col_labels)}")
else:
    print(f"- Итоговый выбор: {positions_to_text(nash_strict, pairs, row_labels, col_labels)}")
print()

print_mixed_analysis(a, b, pairs, row_labels, col_labels)

def main() -> None:
    print("ПРОВЕРКА АЛГОРИТМОВ НА ТРЁХ КЛАССИЧЕСКИХ ИГРАХ")
    for game_name, data in CLASSIC_GAMES.items():
        analyze_game(
            game_name,
            data["pairs"],
            row_labels=data["rows"],
            col_labels=data["cols"],
            extra_comment=data.get("comment"),
        )

    print("СЛУЧАЙНАЯ БИМАТРИЧНАЯ ИГРА 10x10")
    print_separator("-")
    random_pairs = generate_random_bimatrix(
        rows=RANDOM_GAME_SIZE,
        cols=RANDOM_GAME_SIZE,
        low=RANDOM_GAME_LOW,
        high=RANDOM_GAME_HIGH,
        seed=RANDOM_GAME_SEED,
    )
    analyze_game("Случайная игра 10x10", random_pairs)

    print("ВАРИАНТ ИЗ ТАБЛИЦЫ Л5.1")
    print_separator("-")
    analyze_game(
        f"Вариант 13",
        [[(4, 1), (6, 2)], [(11, 7), (0, 5)]],
        row_labels=[" $\alpha_1$ ", " $\alpha_2$ "],
        col_labels=[" $\beta_1$ ", " $\beta_2$ "],
        extra_comment="Для этой игры дополнительно ищем равновесные ситуации в смешанном расширении.",
    )

if __name__ == "__main__":
    main()

```